

실시간 관제 알람 시스템 설계·구현

신뢰성 확보 → 단일 결정자 → 수평 확장까지의 개발 흐름

작성자 최재웅

작성일 2026-06-09

대상 IoT 가스·전력 통합 관제 플랫폼

Redis Stream

단일 결정자

AI mute

fan-out

Discord 연동

목차

- 1 초록
- 2 시스템 개요와 통합 타임라인
- 3 설계 철학 – 2원칙
- 4 개발 흐름 (시간순·결정 중심)
- 5 전력 트랙과의 접합부
- 6 성능·검증
- 7 의사결정과 고민의 흔적
- 8 한계와 향후 과제
- 9 별첨 – 흐름도·근거 문서

문체: 두괄식·"-했음"체. 모든 주장에 코드·수치·테스트 증빙을 붙였음. 날짜는 plan 머리말이 아니라 실제 git 머지일(commit/PR date) 기준임.

1 초록

ABSTRACT

알람 시스템은 "새거나·늦거나·중복되던" 신뢰성 결함을 먼저 잡고, 그 위에 AI 결합·운영자 관점·수평 확장을 차례로 엮는 순서로 개발했음. 4월 기본 골격(AlarmRecord/Event 자동생성·WS 팝업) 위에서, 5/12 큐를 프로세스 메모리에서 Redis로 옮겨 유실·중복·DB 락을 제거했고(#45), 전력 AI를 결합하자마자 드러난 AI-정적을 동시발화 충돌을 `ai_fired:*` 키(AI mute)로 봉합했음(#56). 이후 정보 통일·중복 정리·단일 결정자(T1·T6·T3·T4)로 운영자 관점을 세우고, 6/2 큐를 Redis Stream으로 전환해 작업자·Discord까지 도달 범위를 확장했음(#107·#110). 핵심 설계 동기는 일관되게 "AI는 주역, 정적률은 안전망, 운영자는 알람의 출처를 추적할 수 있어야 한다"임. 결과로 알람 태스크 p95를 460ms→49ms로 89% 단축했고, 핵심 기능 12종을 머지 완료했음.

2 시스템 개요와 통합 타임라인

알람은 다수 진입원(가스·전력·지오펜스·전력 AI)에서 출발해 하나의 Redis Stream으로 모인 뒤 3채널(관리자·작업자·Discord)로 fan-out 되는 구조임.

가스·전력·지오펜스는 `DRF→Celery→DB→push` 경로를, 전력 AI는 지연을 줄이기 위해 `fastapi 직접 push` (Celery 우회) 경로를 사용했음. 두 경로의 공통 종착점이 Redis Stream(`diconai:ws:alarms`)이며, `alarm_flush_loop`가 XREAD로 읽어 브로드캐스트했음.



그림 1. 다-진입원 → Redis Stream 단일 종착 → 3채널 fan-out 구조 (#107 반영)

2.1 통합 타임라인

날짜	단계	핵심	PR
04-15~30	알람 골격 구축	alerts/geofence 앱, AlarmRecord/Event 자동생성, WS 팝업, 이벤트 패널	#1~#33
05-12	신뢰성 Phase 1 ★	큐 메모리 list→Redis LIST(BRPOP), SQLite WAL, dedupe 원자화, push retry	#45
05-15	AI mute + cooldown ★	AI+를 동시발화 봉합: <code>ai_fired:*</code> 60s 키, fingerprint dedup, cooldown 3600→60s	#56
05-15~18	재설계 Phase 1:2 + D옵션	user-scoped ack, RESOLVED auto-close, 모든 페이지 팝업, 미확인 배치·1분 격상	#60·#61·#64
05-19~20	T1+T6+T3 통합 ★	정보 통일(ML용어 제거)·메시지 표준화-dedup 4레이어 정리	#70
05-20~21	T4 단일 결정자 ★	<code>decide_alarm</code> (AI 5상태 × 정적 → source) — AI vs 를 단일 결정	#75

05-22~27	누락/중복 종합·정책 관리·격상 bypass	유실·중복 픽스, 정책 CRUD+권고조치 DB, Celery 큐 분리	#77-#80-#83-#94
06-02	큐 Redis Stream 전환 + 확장 ★	LIST/BRPOP→Stream/XADD+replica별 XREAD, 작업자 DANGER, Discord	#107
06-02	전력 알람 폭주 해소 ★	채널-aware clear + danger 2틱 confirm + confirm 계측	#110
06-05	부하 병목 개선	부하테스트 3종 + Grafana, 큐 LLEN→XLEN	#113-#114

★ = 핵심 마일스톤. plan 문서의 "시연 후 D+30" 머리말은 다수 stale이며, 실제로는 시연 전(05-18~19)에 앞당겨 실행됐음 – 시점은 commit date를 진실로 봄.

3 설계 철학 – 2원칙

알람 발화의 모든 결정은 두 원칙으로 수렴했음.

원칙	내용	구현 장치
① 단일 결정자	발화 결정을 한 곳에 모아 race를 제거. 같은 위험에 여러 경로가 제각기 울리지 않게 함.	<code>decide_alarm</code> (T4), Lua <code>try_transition</code>
② 정적률 = 최종 안전망	시가 침묵·실패·워밍업·장애 상태여도 정적 임계률이 책임지고 발화. AI는 주역, 불은 그늘.	AI mute <code>ai_fired:*</code> , <code>is_ai_mute_active</code> 가드

이 두 원칙은 단순 코딩 규칙이 아니라 "운영자가 알람을 신뢰하려면 누가·왜 울렸는지 추적 가능해야 한다"는 관제 도메인 요구에서 나왔음. 그래서 알람마다 `source` (ai/static_*)와 `algorithm_source` (어느 AI 축)를 라벨로 붙여 출처를 노출했음.

4 개발 흐름 (시간순·결정 중심)

기능을 더 없기 전에 그릇(신뢰성)부터 튼튼히 한 뒤, 충돌을 봉합하고, 운영자 관점으로 정리하고, 마지막에 수평 확장한 순서임.

4.1 알람 골격 구축 2026-04-15 ~ 04-30 · PR #1~#33

관제 기본 골격으로 `alerts`·`geofence` 앱을 세우고, 위험 판정을 불변 기록으로 남기는 `AlarmRecord`와 워크플로우를 다루는 `Event`를 분리해 자동 생성했음. WebSocket 팝업과 이벤트 패널, 지오펜스 위치 연동까지 1차 구현했음.

이 단계에서 판정(`AlarmRecord`)과 운영(`Event`)의 책임 분리를 정해, 이후 ack-RESOLVED-dedup 설계가 모두 이 두 모델 위에 얹혔음.

4.2 신뢰성 Phase 1 — 큐를 메모리에서 Redis로 2026-05-12 · PR #45

기능을 더 없기 전에, 알람이 새거나·늦거나·중복되거나·DB가 잠기던 신뢰성 결함을 먼저 잡았음.

AS-IS (문제)

프로세스 메모리 `list` + `asyncio.Event` 큐. 재시작 시 휘발, race 발생, `[ :5]` cap으로 silent drop. SQLite 다중 writer가 `database is locked` 폭주. Celery retry가 같은 알람을 3중 발송.

TO-BE (조치)

큐를 Redis LIST(LPUSH/BRPOP)로 이관(휘발·race 제거). SQLite WAL 적용. dedupe를 Lua `try_transition` 으로 원자화. `bulk_create` 정합성 확보. `_push_to_ws` retry 추가.

증빙 PR #45(C1~C5 커밋). 근거: 2026-05-12_[plan]알람-신뢰성-Phase1.md, [변경]알람-신뢰성-Phase1-변경요약.md. 큐 전환 효과는 \$6에서 부하·락 수치로 교차 검증.

4.3 AI mute + cooldown — 접합부의 탄생 2026-05-14~15 · PR #56

전력 AI를 알람에 묶자마자 AI와 정적률이 같은 위험에 둘 다 올리는 충돌이 즉시 터졌고, 이를 Redis 키로 1차 봉합했음.

AS-IS

전력 IF 추론 결과를 AlarmRecord로 묶자, AI(**POWER_ANOMALY_AI**)와 DRF 정적률이 동시 발화. 위험이 **CACHE_TTL=3600** 으로 1시간 정적 고정. Celery retry로 push 3중복.

TO-BE

push fingerprint dedup(SET NX EX). AI mute 키 **ai_fired:{device}:{ch}:{level}** 60s → DRF 를 태스크가 **is_ai_mute_active** 로 자기 발화를 억제. cooldown 3600→60s.

이 **ai_fired:*** 키는 이후 알람·전력 두 트랙을 잇는 핵심 접합부가 됐음(\$5에서 상술). "AI=주역, 룰=안전망"을 코드로 강제하는 첫 장치임.

증빙 PR #56. 근거: **2026-05-15_[변경]알람-AI뮤트+쿨다운-Phase2.md**.

4.4 알람 재설계 Phase 1·2 + D옵션 2026-05-15 ~ 05-18 · PR #60·#61·#64

"알람이 뜬다"에서 "여러 페이지·여러 탭·자리비움 상황에서도 빠짐없이 도달한다"로 도달성을 끌어올렸음.

AS-IS

ack가 전역이라 한 운영자가 확인하면 모두 사라짐. RESOLVED가 자동 정리 안 됨. 모니터링 외 페이지에선 팝업 미표시. 재연결 시 놓친 알람 catch-up 없음. 자리비움/다중탭에서 중복.

TO-BE

user-scoped ack(**EventAcknowledgement**), RESOLVED auto-close, WS catch-up(**since=**). 공통 **alarm_stack** 으로 모든 페이지 팝업·admin-panel 토스트/격상 모달. 미확인 배지·클라 60s dedup·1분 격상, **algorithm_source** 필드(마이그 0018).

증빙 PR #60/#61/#64. 근거: **alarm-system-redesign**, **alarm-phase2-completion**, **D옵션-미확인배지+클라dedup+토스트격상**. ※ "D옵션"은 2도메인별 결정과 다른, 2헤더 배지+60s dedup+1분 격상을 가리킴(인용 시 구분).

4.5 운영자 관점 정리 — T1·T6·T3 2026-05-19 ~ 05-20 · PR #70

as-is 분석에서 도출한 29개 갭을 7개 테마(T1~T7)로 재구성하고, 시연 전 **T1·T6·T3·T4**를 우선 채택했음.

테마	문제(AS-IS)	조치(TO-BE)
T1 정보 통일	운영자 화면에 ML 용어 노출, message/summary 이원화	get_short_message 단일 공급원으로 통일, 한글 워딩. ※ AI 출처 칩은 운영자 검토 후 revert 제거("출처는 엔지니어 채널로")
T6 메시지 표준화	도메인마다 제각각 문구	경상화 메시지 통일, 도메인 메시지 양식 표준화
T3 dedup 통일	4개 dedup 레이어가 분산, RESOLVED가 dedup에 막힘	4레이어 의도 문서화, EventAck "(N 확인 중)"(event_ack_users), RESOLVED key 분리 (:resolved), TTL 60s 유지

증빙 T1·T6·T3는 같은 브랜치로 #70 통합 머지. 근거: **T1+T6-운영자정보통일**, **T3-중복방지dedup통일**.

4.6 단일 결정자 decide_alarm — T4 2026-05-20 ~ 05-21 · PR #75

"AI가 잡았나 / 룰이 잡았나"를 한 곳에서 확정하는 단일 결정자를 두어, 원칙 ①을 코드로 못박았음.

AS-IS

AI(**POWER_ANOMALY_AI**)와 정적률(**POWER_OVERLOAD**)이 병렬 동시 발화. DRF가 직접 push하는 경로가 산재해 결정 지점이 분산.

TO-BE

fastapi **decide_alarm** 이 AI state 5종 × 정적 평가를 매트릭스로 받아 단일 source 산출. DRF 직접 push 경로 정리, **source / reason** payload + 마이그, 프론트 source별 시각 분기.

원래 부모 plan은 T4를 "시연 후(우선순위 6)"로 잡았으나, "AI는 주역·정적률은 안전망"이라는 정체성 자체가 시연 가치라 판단해 05-20~21로 앞당겨 구현했음.

우선순위 고민 잔존 작업(T2 4색상·T5 모달 정책·T7 JS 인프라)보다 T4를 앞세운 건 의도적 선택이었음 — 화면을 더 꾸미는 것보다 "누가 왜 올렸나"를 한 곳에서 확정하는 구조가 시연에서 더 설득력 있다고 봤음. 대신 T2·T5·T7은 시연 후로 미뤄 일정 리스크를 분산했음.

증빙 PR #75(T4 D1a~D4). 근거: T4-D2-단일정책결정-구현스펙, decide_alarm-T4단일결정매트릭스 코드리뷰.

4.7 큐 Redis Stream 전환 + 작업자·Discord 확장 2026-06-01 ~ 06-02 · PR #107

멀티레플리카를 대비해 큐를 Redis Stream으로 바꾸고, 같은 머지에서 알람 도달 범위를 작업자·Discord까지 넓혔음.

AS-IS

BRPOP은 경쟁 소비라 replica가 2개 이상이면 한쪽만 받고 누락. 작업자 WS는 지오펜스만 받아 가스/전력 DANGER 미수신. 외부 통보 채널 없음.

TO-BE

LIST/BRPOP → Stream/XADD(MAXLEN~10000) + replica별 독립 XREAD(Consumer Group 아님 = fan-out), alarm_stream_lag 메트릭. 작업자 개인 fan-out(worker_clients), Discord 연동(관리자 broadcast / 작업자 @here 대피 멘션).

왜 지금·왜 Consumer Group이 아닌가 시연을 코앞에 두고 큐를 갈아끼우는 건 위험한 선택이라, "동작 변화 0"을 먼저 못박고 dedup-격상 로직을 그대로 둔 채 전송 계층만 바꿨음. Consumer Group(경쟁 소비)이 아니라 replica별 독립 XREAD(fan-out)를 택한 건, 모든 replica가 같은 알람을 받아 각자의 WS 연결로 뿌려야 하는 broadcast 특성 때문임 — 작업 분배가 아니라 복제가 목적이라 의도적으로 Consumer Group을 피했음.

주의 큐 전환은 동작 변화 0(시연 영향 없음)이고, 동기는 순수하게 수평 확장 대비임. dedup(SET NX EX fingerprint)은 LIST-Stream 내내 동일 유지. plan 헤더 "구현 대기"는 stale이며 #107로 완료됨.

증빙 PR #107. 근거: 알람Stream전환-1~4 (규계층/소비루프/배포안전장치/E2E검증), Discord알람연동-asbuilt.

4.8 전력 알람 폭주 해소 2026-06-02 · PR #110

AS-IS

clear가 device-wide로 동작해 채널 하나만 변해도 전체 채널이 RESOLVE churn → 팝업 연쇄 폭주.

TO-BE

채널-aware clear(마지막 활성 채널만 RESOLVE) + danger 2틱 confirm(단발 노이즈 억제) + confirm 계측.

증빙 PR #110. 근거: 전력알람폭주-팝업연쇄-수정, 전력알람-폭주+팝업cadence 진단.

5 전력 트랙과의 접합부

알람과 전력 AI는 독립이 아니라 세 개의 접합부로 묶였음. 전력의 "어느 축이 잡았나"가 알람의 "운영자 추적"으로 연결되는 지점임.

접합부	역할	도입
① AI mute ai_fired:{device}:{ch}:{level} TTL 60s	전력 AI가 발화하면 fastapi가 Redis에 마킹 → DRF 를 태스크가 is_ai_mute_active 로 자기 발화 억제. "AI=주역, 룰=안전망"을 강제하는 키.	05-15 #56
② decide_alarm 단일 결정자	전력 AI state(5종) × 정적 평가 → 알람 source 5종 산출. 전력 추론 파이프라인의 마지막 단계이자 알람 발화의 시작점.	05-20~21 T4

③ <code>algorithm_source</code> 6종 라벨	전력 5축(night/combined/change_point/arima/zscore/IF) 중 무엇이 잡았는지를 알람 라벨로 전달.	재설계~T4
--	---	--------

용어 주의 `source` (ai / static_*) ≠ `algorithm_source` (어느 AI 축). `decide_alarm` 이 내는 source는 5종, `AlarmSource` enum은 6종 (+ `static_legacy` = DRF 롤/backfill). 인용 시 둘을 혼동하지 말 것.

6 성능·검증

알람 성능은 "충분히 검증했고, 개선 방향은 동시성·병렬화"라는 선에서 간략히 다룸. 정량 수치는 측정 시점과 evidence 출처를 함께 명기했음.

6.1 알람 성능 개선 실측 (PR #114)

지표	before	after	근거
Celery alarm 태스크 p95	460ms	49ms (↓89%)	concurrency 2→4 (<code>docker-compose.yml</code> : 162)
<code>_send_to_all</code> broadcast	순차	<code>asyncio.gather</code> 병렬 → <code>alarm_stream_lag</code> 0 유지	<code>ws_router.py</code> :51
가스 DRF 저장	동기(루프 블로킹)	비동기 분리 → 이벤트 루프 블로킹 제거	<code>gas_service.py</code>

인용 주의 p95 460→49ms는 커밋 메시지 출처로, Grafana/Celery task duration 1차 evidence는 미첨부임. 코드 자체(`--concurrency=4`)는 확정. 발표 시 "커밋 기록 기준, evidence 별도 캡처 필요"로 표시할 것.

6.2 알람 3대 증상 병목 진단 (UX·도달성)

2026-05-20 진단은 성능 수치가 아니라 도달성/UX 병목임 – 정량 성능치로 인용하지 말고 트러블슈팅 서사로만 사용했음.

증상	1순위 원인	방향
A 모델 안 뜰/새로고침해야 옴	<code>_AckStore</code> 24h localStorage 영구 차단	TTL 24h→60s / RESOLVED 동기 삭제
B AI+를 중복·재연결 누락	AI mute 60s 만료 후 둘 다 발화 + catch-up이 클라 dedup에 차단	catch-up cap + bypass 플래그
C 조치완료해야 다음 알람	ack(user-scoped) vs RESOLVED(전역) 멘탈모델 충돌(코드 버그 아님)	UX 통합(T5 sprint)

6.3 관측 가능성 – `alarm_stream_lag`

Stream 말단 ID와 소비 커서의 시간차(초)를 `alarm_stream_lag` 로 계측해, 평상시 ≈0을 유지하고 replica 뒤처짐을 식별했음(#107). 큐 깊이 지표도 LIST `LLEN` → Stream `XLEN` 으로 전환했음.

6.4 부하·처리량 맥락

알람 경로의 broadcast 병렬화·concurrency 4 효과는 06-05 부하테스트 after 수치에는 미반영임 – 측정(#113 직후)이 #114 머지 이전이라 그러함. 따라서 알람 성능 개선은 §6.1 태스크 단위 수치로만 인용하고, 시스템 처리량은 전력/인프라 문서의 측정시점 캐비닛과 함께 봐야 함.

7 의사결정과 고민의 흔적

알람 작업의 상당 부분은 코드를 짜기 전에 "지금 무엇이 문제이고, 무엇부터 손대야 하는가"를 정리하는 분석이었음. 결과만이 아니라 그 판단 과정을 남김.

7.1 29개 갭 → 7개 테마로 재구성

재설계 직후(05-19), AI 추론부터 토스트 격상까지 전 경로의 As-Is를 한 장에 명시화하고, "의도 vs 실재가 어긋나는 후보 갭"을 29개 뽑았음. 이를 그대로 처리하면 산만해져서, 성격이 같은 것끼리 7개 테마(T1~T7)로 묶고 시연 가치 순으로 우선순위를 매겼음.

테마	다른 문제	시연 전 여부
T1 정보 통일	ML 용어 노출·message/summary 이원화	☑ 완료 (#70)
T6 메시지 표준화	도메인별 제각각 문구	☑ 완료 (#70)
T3 dedup 통일	4 레이어 분산·RESOLVED 차단	☑ 완료 (#70)
T4 단일 결정자	AI vs 톨 병렬 발화	☑ 완료 (#75)
T2·T5·T7	4색상 위험도·모달 이탈정책·JS 인프라	☑ 시연 후

왜 이 순서 "정보 통일(T1·T6) → 중복 정리(T3) → 단일 결정(T4)"로 진행한 건, 운영자가 신뢰하려면 먼저 말이 일관돼야 하고, 그 다음 중복이 없어야 하고, 마지막에 출처가 하나로 확정돼야 한다는 순서라고 봤기 때문임. 시각적 꾸밈(T2 4색상)은 이 논리 위에 얹는 것이라 뒤로 미뤘음.

7.2 되돌린 결정 – T1 AI 출처 칩

처음 설계

알람마다 "AI가 잡음 / 톨이 잡음"을 알려주는 출처 칩을 운영자 화면에 노출하려 했음.



되돌림

운영자 검토 결과 "AI 출처는 운영자가 아니라 엔지니어가 볼 정보"라는 피드백을 받아 칩을 **revert**로 제거했음. 출처 추적은 **algorithm_source** 라벨로 백엔드·엔지니어 채널에만 남겼음.

배운 것 "추적 가능하게 만든다"와 "운영자에게 보여준다"는 다른 문제였음 – 정보를 만들어 그 정보를 누구에게 노출할지는 별도 결정이라는 걸, 만들고 나서 되돌리며 배웠음.

7.3 값 하나에도 트레이드오프 – rate limit / cooldown

같은 위험의 재발화를 막는 쿨다운 값을 정할 때 30s(시연 가시성 높음) ↔ 60s ↔ 120s(폭주 완전 차단) 사이에서 저울질했음. 최종 60s는 "폭주 회피 > 시연 가시성"이라는 명시적 우선순위와, 전력 AI mute(**ai_fired.***)의 TTL 60s와 키 수명을 맞춰 두 장치가 어긋나지 않게 하려는 의도의 결과임. 다만 위험도가 상승할 때는 쿨다운을 우회(#94)해, "조용히 묻히는" 위험을 막았음.

7.4 일부러 미룬 것 – 작업자 수동 알람

운영자가 작업자에게 직접 통보하는 기능은 구현하지 않고 보류했음. 기능이 어려워서가 아니라, "작업자 디바이스에서 어떤 UX로 받을지"가 먼저 정해져야 알람 형태(인앱/푸시/멘션)가 결정되기 때문임. 디바이스 결정이 선행되지 않은 상태에서 만들면 다시 갈아엎을 가능성이 커서, 의도적으로 Phase 3로 미뤘음.

정직성 노트 §6.2의 3대 증상 진단(모달 미표시·AI+를 중복·ack 멘탈모델)은 "검증된 성능 수치"가 아니라 도달성·UX 병목임. 이런 약점도 숨기지 않고 진단서 사로 남겨, 무엇이 아직 거친지를 분명히 했음.

8 한계와 향후 과제

시연 대상 핵심(Stream·작업자·Discord·단일 결정·폭주 해소·dedup)은 전부 완료했고, 잔존은 멀티레플리카와 디바이스 의존 항목임.

항목	내용	상태
fastapi 멀티레플리카	Redis 공유상태 + pub/sub. AI 시계열 윈도우 공유가 유일 블로커(계층 2). 알람 전송계층은 Stream fan-out으로 이미 준비됨.	☑ 시연 후
작업자 수동 알람	운영자가 작업자에게 직접 통보 – 디바이스 UX 결정 선행 필요(미구현 추정)	☑ Phase 3
알람 T2·T5·T7	4색상 위험도(T2), 모달 이탈정책 일부(T5), JS 인프라(T7)	☑ 시연 후
_AckStore 멘탈모델	ack vs RESOLVED UX 통합 패치	☑ 일부

문서 인용 주의: "Phase 1/2"는 신뢰성(#45)과 재설계(#60/#64) 두 맥락에서 같은 이름을 쓰므로 구분할 것. stream/discord/작업자 DANGER의 plan 헤더 "구현 대기"는 #107로 완료된 stale 표기임.

9 별첨 — 흐름도·근거 문서

9.1 알람 큐 3단계 전환



그림 2. 알람 큐 진화 — 메모리 → Redis LIST → Redis Stream

9.2 중복 차단 3계층 + AI mute

알람 중복은 네 겹으로 막았음: ① `try_transition` (Lua 원자 상태천이) ② Event 쿨다운 60s(격상 시 우회) ③ fingerprint dedup(SET NX EX 30s, 4분기) ④ AI mute(`ai_fired:*` 60s, 격상 bypass). 각 레이어의 의도를 T3에서 문서화해 RESOLVED가 dedup에 막히지 않도록 key를 분리했음.

9.3 근거 문서 인덱스

구분	파일명	비고
종합 SoT	alarm/2026-06-08_[인덱스]알람-작업-마스터인덱스.md	마스터 인덱스
종합 SoT	알람-자료인덱스.md	자료 인덱스
최신 E2E	alarm-e2e-trace.md	4진입원 fan-out, #107 · #110 반영
신뢰성	2026-05-12_[plan]알람-신뢰성-Phase1.md	
AI mute	2026-05-15_[변경]알람-AI뮤트+쿨다운-Phase2.md	
재설계	2026-05-1 람시스템재설계-최종구현계획.md	
T1~T7	2026-05-19_[plan]알람-비즈니스로직재정립-T1~T7.md	
as-is 29갯	2026-05-19_[진단]알람-비즈니스로직-현황분석.md	
T4	2026-05-20_[spec]T4-D2-단일정책결정-구현스펙.md	
Stream	2026-06-01_[study]알람Stream전환-1~4.md	
Discord	2026-06-01_[변경]Discord알람연동-asbuilt.md	
폭주	2026-06-02_[plan]전력알람폭주-팝업연쇄-수정.md	
성능	alarm/성능검증-알람.md	

본 문서의 모든 시점은 git 머지일 기준이며, plan 머리말의 “시연 후 D+30” 표기는 다수 stale임. 수치는 코드를 truth source로 대조했고, 과대서술 없이 “한 일을 또렷하게” 기술하는 것을 원칙으로 삼았음. — 최재용, 2026-06-09